A Hybrid Regex-BERT-LLM Framework for Automated Log Classification in Security Operations Centers

[1]Mr.S.Gowtham
Assistant Professor
Department of Computer Science and Engineering
Jai ShriramEngineering College
Tiruppur,Tamil Nadu, India
email: ssgowtham1996@gmail.com

[2] M Paranjothi Karthik
PG Student
Department of Computer Science and Engineering
Jai ShriramEngineering College
Tiruppur,Tamil Nadu, India
email: paranjothikarthik2003@gmail.com

***Abstract*—Security Operations Centres (SOCs) generate and process massive volumes of heterogeneous and unstructured system logs, making real-time classification a critical yet challenging task for effective threat detection and response. Traditional rule-based approaches such as regular expressions (Regex) provide high precision and low latency for known patterns but fail to generalize to unseen or complex logs. In contrast, deep learning models such as Bidirectional Encoder Representations from Transformers (BERT) offer strong contextual understanding but incur higher computational costs. Recent advancements in Large Language Models (LLMs) further enhance semantic reasoning capabilities, although they introduce additional latency and scalability concerns This paper proposes a Hybrid Regex–BERT–LLM framework for automated log classification that integrates deterministic pattern matching, contextual deep learning, and semantic reasoning into a unified multi-stage pipeline. The system employs a three-tier architecture consisting of a Regex-based fast filtering layer, a fine-tuned BERT model for contextual classification, and an LLM-based fallback mechanism for handling complex or previously unseen logs. A confidence-driven decision strategy dynamically routes log messages across stages to optimize both accuracy and computational efficiency. Experimental evaluation on a dataset of 2,410 SOC logs spanning nine categories demonstrates an overall classification accuracy of 96.4%, with an average confidence score of 93.1% and a processing latency of 62.01ms per log. The results indicate improved robustness and scalability compared to standalone approaches, making the proposed framework suitable for real-world SOC automation workflows.**

***Keywords*—** *Log Classification, Security Operations Center (SOC), Regular Expressions (Regex), Bidirectional Encoder Representations from Transformers (BERT), Large Language Models (LLMs), Hybrid Artificial Intelligence, Cybersecurity Automation, Log Analysis, Natural Language Processing (NLP), Incident Detection*

I. Introduction

Modern Security Operations Centres (SOCs) continuously monitor and analyse large volumes of system-generated logs across complex and distributed environments. With the rapid growth of cloud computing and microservices, log data has become highly heterogeneous and unstructured, making automated classification a challenging yet essential task for effective threat detection. Traditional rule-based approaches, particularly regular expressions (Regex), are widely used due to their efficiency and precision in detecting known patterns. However, they lack the ability to generalize to unseen or context-dependent logs, resulting in reduced effectiveness in dynamic security environments.

To address these limitations, machine learning approaches, especially transformer-based models such as Bidirectional Encoder Representations from Transformers (BERT), have been adopted for log classification. These models capture contextual semantics and improve classification of ambiguous logs but introduce challenges related to computational cost and inference latency. More recently, Large Language Models (LLMs) have demonstrated strong capabilities in semantic reasoning and log interpretation, although their high resource requirements limit their use in real-time, high-throughput systems.

This paper proposes a Hybrid Regex–BERT–LLM framework that integrates deterministic pattern matching, contextual deep learning, and semantic reasoning within a unified multi-stage pipeline. The system consists of a Regex layer for fast classification of structured logs, a fine-tuned BERT model for context-aware classification, and an LLM-based fallback for complex or unseen logs. A confidence-driven decision mechanism dynamically routes logs across stages to optimize both accuracy and efficiency.

Experimental evaluation on 2,410 log entries across nine categories demonstrates that the proposed framework achieves high classification accuracy with low latency. The results highlight its robustness, scalability, and suitability for real-time SOC automation workflows, outperforming standalone approaches.

II. Literature Review

The increasing volume and complexity of system logs in modern computing environments have led to extensive research in automated log analysis, particularly within Security Operations Centres (SOCs). Existing approaches can be broadly categorized into rule-based methods, machine learning and deep learning techniques, and recent large language model (LLM)-driven frameworks.

A. Rule-Based Log Analysis

Early log classification systems primarily rely on rule-based techniques such as regular expressions (Regex) to identify predefined patterns in log messages. These methods are widely adopted in SOC pipelines due to their low computational overhead and deterministic behavior. Regex-based systems are effective for structured logs, including HTTP status codes, system alerts, and known error signatures. However, their performance degrades significantly when handling unstructured or previously unseen logs, as they lack contextual understanding and require continuous manual rule updates. This limitation motivates the integration of learning-based approaches for improved generalization.

B. Maintaining the Integrity of the Specifications

To overcome the rigidity of rule-based systems, machine learning models have been applied to log classification and anomaly detection tasks. Traditional approaches utilize statistical and probabilistic methods to model log distributions, while more recent work leverages deep learning architectures.

Recurrent neural networks, particularly Long Short-Term Memory (LSTM) models, have been employed for sequence-based log anomaly detection. For instance, LSTM-based frameworks model temporal dependencies in log sequences to identify deviations from normal behaviour. While effective for sequential anomaly detection, these models often struggle with large-scale, heterogeneous log data and require extensive preprocessing.

Transformer-based models such as BERT have significantly advanced log analysis by enabling contextual understanding of unstructured text. BERT-based classifiers leverage bidirectional attention mechanisms to capture semantic relationships within log messages, improving classification accuracy for ambiguous and noisy inputs. However, these models introduce higher computational costs and latency, making them less suitable for real-time SOC environments without optimization.

C. Log Summarization and Semantic Understanding

Recent studies have explored log summarization as a means to improve human interpretability and incident response. Systems such as *Log Summary* and other NLP-based approaches focus on transforming unstructured logs into concise natural language summaries. Similarly, research on automated narrative generation for cybersecurity logs aims to provide higher-level insights into system behaviour and attack patterns.

The *CYGENT* framework introduces a conversational agent for cybersecurity that leverages generative AI to summarize logs and assist analysts. While such systems enhance usability and decision-making, their primary focus is on summarization rather than high-throughput classification. Additionally, they do not address the need for efficient multi-stage processing pipelines required in real-time SOC operations.

D. Large Language Models in Cybersecurity

Large Language Models (LLMs) have recently gained attention for their ability to perform complex reasoning and semantic interpretation tasks. In cybersecurity, LLMs have been applied to log summarization, anomaly explanation, and incident report generation. These models can handle highly unstructured and previously unseen data, making them valuable for advanced analysis.

LLMs are computationally expensive and introduce significant latency, limiting their direct applicability in large-scale log processing pipelines. As a result, their role is often restricted to post-processing tasks rather than primary classification. Efficient integration of LLMs into real-time systems remains an open research challenge.

E. Research Gap and Motivation

Despite significant progress, existing approaches exhibit several limitations:

- Rule-based systems lack adaptability and fail on unseen patterns.
- Deep learning models improve accuracy but introduce latency and resource overhead.
- LLM-based systems provide semantic richness but are not optimized for real-time classification.
- Existing frameworks, including conversational SOC systems, do not effectively combine these approaches into a unified, scalable pipeline.

To address these challenges, there is a need for a hybrid framework that leverages the strengths of rule-based methods, contextual deep learning, and semantic reasoning while mitigating their individual limitations.

F. Positioning of the Proposed Work

In contrast to existing approaches, the proposed Hybrid Regex–BERT–LLM framework integrates deterministic pattern matching, contextual classification, and semantic fallback into a single pipeline. Unlike prior work that focuses on either classification or summarization, this approach introduces a confidence-driven orchestration mechanism to dynamically select the most efficient and accurate processing stage. This enables improved scalability, robustness, and real-time applicability in SOC environments.

III. Problem Statement and Research Gap

A. Problem Statement

Security Operations Centers (SOCs) are required to process and analyze large volumes of system-generated logs originating from diverse sources such as servers, applications, network devices, and cloud services. These logs are inherently highly heterogeneous, unstructured, and continuously evolving, making automated classification a non-trivial task. Accurate log classification is essential for downstream operations including incident detection, alert prioritization, threat correlation, and automated ticket generation. In practical SOC environments, log classification systems must simultaneously achieve high accuracy to correctly identify security-relevant events, low latency to support real-time or near real-time processing,

and scalability to handle high-throughput log streams. However, achieving these requirements concurrently remains a significant challenge. Existing approaches exhibit inherent trade-offs: rule-based methods such as Regex provide fast and deterministic classification but fail to generalize to unseen or complex log patterns; deep learning models such as BERT offer strong contextual understanding but incur higher computational cost and latency; and Large Language Models (LLMs) enable advanced semantic reasoning but are not optimized for real-time, high-volume inference due to resource constraints. As a result, current SOC pipelines either sacrifice accuracy for speed or incur significant latency to achieve improved classification performance, limiting their effectiveness in real-world security automation workflows.

B. Research Gap

A critical analysis of existing literature and systems reveals several important research gaps. First, most existing approaches focus on a single paradigm—rule-based, machine learning, or LLM-based—without effectively integrating their complementary strengths into a unified framework. Second, current systems lack adaptive or confidence-based decision mechanisms to dynamically determine which model should process a given log message, resulting in inefficient resource utilization. Third, there is limited emphasis on real-time SOC constraints, as many solutions prioritize accuracy or interpretability while overlooking latency, throughput, and scalability requirements essential for operational environments. Furthermore, existing approaches inadequately handle unseen and ambiguous logs; rule-based systems fail on novel patterns, while standalone deep learning models may produce uncertain predictions without a semantic fallback mechanism. Finally, most works treat log classification, storage, and incident response as independent components, lacking an end-to-end integrated pipeline that supports automated workflows such as database persistence and ticket generation. These limitations collectively highlight the need for a more robust, adaptive, and scalable solution.

C. Research Objective

To address these gaps, this work aims to design and implement a Hybrid Regex–BERT–LLM framework that:

- Combines speed (Regex), contextual accuracy (BERT), and semantic reasoning (LLM) in a unified architecture.
- Introduces a confidence-driven decision mechanism to dynamically select the most appropriate processing stage.
- Ensures low-latency and scalable performance suitable for real-time SOC applications.
- Provides an end-to-end automated pipeline integrating log classification, storage, analytics, and incident management.

D. Key Research Questions

The proposed study is guided by the following research questions:

- *RQ1:* How can rule-based, deep learning, and LLM-based approaches be effectively combined to improve log classification performance?
- *RQ2:* Can a confidence-driven multi-stage pipeline reduce latency while maintaining high accuracy?
- *RQ3:* How does the hybrid framework perform compared to standalone Regex and BERT models in real-world SOC scenarios?
- *RQ4:* Can the proposed system generalize to unseen and complex log patterns without significant performance degradation?

IV. Proposed Methodology

A. Overview of the Proposed Framework

This paper proposes a Hybrid Regex–BERT–LLM framework for automated log classification in Security Operations Centres (SOCs). The system is designed as a multi-stage, confidence-driven pipeline that integrates deterministic pattern matching, contextual deep learning, and semantic reasoning to achieve high accuracy with low latency.

The overall workflow consists of three sequential processing stages:

Stage 1: Regex-Based Fast Filtering
Stage 2: BERT-Based Contextual Classification
Stage 3: LLM-Based Semantic Fallback

A confidence-based decision mechanism dynamically determines whether a log should be classified at the current stage or passed to the next stage for further analysis. This design ensures optimal utilization of computational resources while maintaining classification robustness.

B. System Architecture

The proposed system follows a modular and scalable architecture, as implemented in the HyBERT-SOC pipeline. The key components include:

- Input Layer: Ingestion of raw log messages from CSV datasets or real-time streams.
- Processing Engine: Multi-stage classification using Regex, BERT, and LLM models.
- Batch Processing Module: High-performance execution using parallel processing (ThreadPoolExecutor).
- Database Layer: Persistent storage using relational databases (SQLite/PostgreSQL).
- Integration Layer: Automated incident creation via REST APIs (e.g., JIRA).
- Visualization Layer: Interactive dashboard for monitoring classification results and analytics.

This architecture supports **end-to-end automation**, from log ingestion to incident management, enabling efficient SOC workflows.

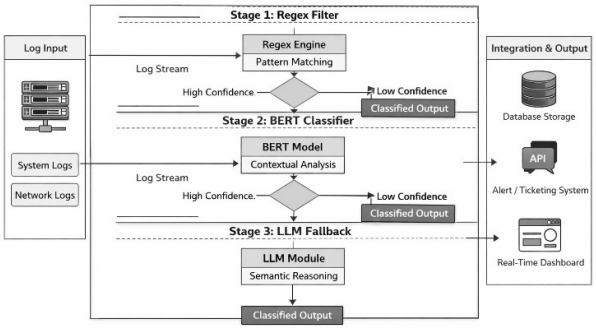


Fig. 1. System Architecture. (*Hybrid framework)*

C. Stage 1: Regex-Based Classification

The first stage applies a set of predefined regular expression patterns to rapidly classify logs that match known structures. This stage is designed to achieve high-speed processing with minimal computational overhead while effectively detecting well-defined patterns such as HTTP errors and authentication failures. Logs that match predefined patterns are immediately assigned a label along with a high confidence score, ensuring efficient handling of structured data. If a log message does not match any pattern, it is forwarded to the subsequent classification stage for further analysis.

D. Stage 2: BERT-Based Contextual Classification

Logs that remain unclassified after the Regex stage are processed using a fine-tuned BERT model trained on labeled log data for multi-class classification across nine categories. This stage leverages context-aware embeddings to effectively handle unstructured and ambiguous log messages. The model is fine-tuned on domain-specific data to enhance classification accuracy and generate class probabilities for confidence estimation. The BERT classifier outputs a predicted label along with an associated confidence score. If this confidence exceeds a predefined threshold, the classification is accepted; otherwise, the log is forwarded to the LLM-based fallback stage for further interpretation.

E. Stage 3: LLM-Based Semantic Fallback

For logs with low-confidence predictions from the BERT model, a Large Language Model (LLM) is employed as a fallback mechanism. This stage is specifically designed to handle complex and previously unseen log patterns by leveraging advanced semantic interpretation and contextual reasoning capabilities. Although LLMs are computationally expensive, their usage is restricted to a small subset of difficult cases, thereby minimizing the overall latency while improving classification robustness and coverage.

F. Confidence-Driven Decision Mechanism

A key contribution of the proposed framework is the confidence-based orchestration strategy that governs transitions between processing stages. The decision process operates by first assigning a label when a Regex match is found with high confidence. In the absence of a match, the log is processed by the BERT model, and its prediction is accepted if the confidence exceeds a predefined threshold. Otherwise, the system invokes the LLM for final classification. This adaptive routing mechanism reduces computational overhead, improves classification reliability, and ensures efficient handling of edge cases.

G. High-Performance Batch Processing

To support large-scale log processing requirements, the system incorporates a parallel batch processing engine implemented using a thread pool. This enables concurrent classification of multiple log messages, significantly improving throughput and reducing overall execution time. The design ensures scalability and makes the system suitable for high-volume SOC environments where rapid processing is critical.

H. Integration with SOC Workflow

Beyond classification, the proposed framework integrates seamlessly with SOC operations to provide a complete automation solution. Classified logs are stored in a database for auditing and analytical purposes, while results are serialized into CSV and JSON formats for further processing. Critical logs can automatically trigger incident creation through JIRA integration, enabling faster response times. Additionally, a Streamlit-based dashboard provides real-time visualization and monitoring of classification outputs. This end-to-end integration transforms the system into a practical SOC automation platform rather than a standalone classification model.

I. Algorithmic Workflow

The overall methodology can be summarized as follows:

1. Input raw log message.
2. Apply Regex-based classification.
3. If no match, pass to BERT classifier.
4. Evaluate BERT confidence score.
5. If confidence is high, assign label.
6. Else, invoke LLM for final classification.
7. Store results and trigger downstream actions.

J. Summary of Methodological Advantages

The proposed methodology effectively combines the speed of rule-based systems, the contextual accuracy of deep learning models, and the semantic understanding of LLMs within a unified framework. By reducing reliance on any single approach and optimizing resource utilization through adaptive routing, the system achieves a balance between accuracy and efficiency. Furthermore, its scalable design and integration with SOC workflows make it well-suited for real-time deployment in practical cybersecurity environments.

V. Dataset Description

A. Dataset Overview

The experimental evaluation of the proposed Hybrid Regex–BERT–LLM framework is conducted using a synthetic Security Operations Center (SOC) log dataset obtained from a publicly available repository. The dataset consists of 2,410 log entries, designed to simulate real-world system and security l ogs generated from diverse sources such as web servers, authentication systems, databases, and network devices. The logs are intentionally varied in structure and complexity to reflect

practical SOC scenarios, including both structured and unstructured formats.

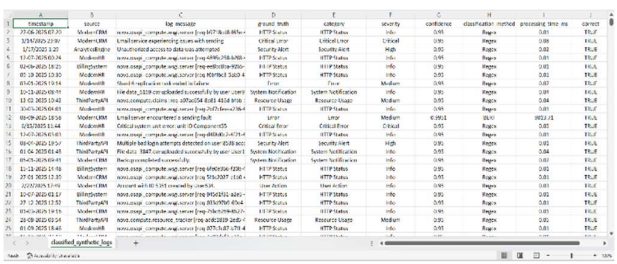
Each log entry is associated with a predefined class label representing a specific type of system event or security condition. The dataset is organized into nine distinct categories, enabling multi-class classification and comprehensive evaluation of the proposed model.

B. Log Categories and Class Distribution

The dataset includes nine log categories that represent common SOC-relevant events, such as critical system failures, authentication issues, network anomalies, and informational messages. These categories provide a balanced mix of normal operational logs and security-critical events, ensuring that the classification model is evaluated across varying levels of severity and complexity.

A summary of the class distribution is presented in Table I, while the overall category distribution is illustrated in Fig. 2 (generated from the experimental results). The dataset is designed to maintain a reasonably balanced distribution across categories to avoid bias during model training and evaluation.

Table I
Log Categories and Class Distribution



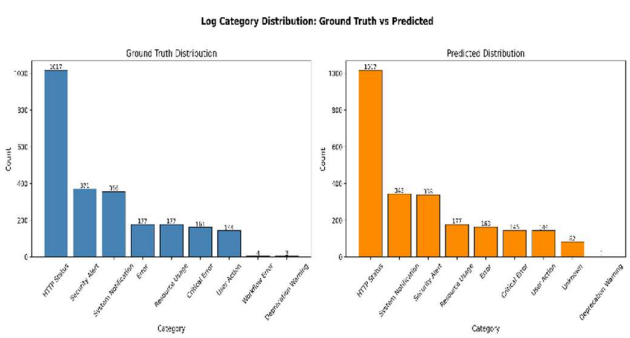


Fig. 2. Category Distribution Chart from paper_results/category_distribution.png (*Dataset analysis)*

C. Log Complexity Levels

To evaluate the robustness of the proposed framework, the dataset incorporates three levels of log complexity:

1. Low Complexity: Structured logs with clear and consistent patterns (e.g., standard HTTP status messages).
2. Medium Complexity: Semi-structured logs with partial variability in format and content.
3. High Complexity: Unstructured or noisy logs with ambiguous semantics and irregular patterns.

This stratification enables the assessment of model performance across different difficulty levels. Regex-based classification is expected to perform well on low-complexity logs, while BERT and LLM components are essential for handling medium- and high-complexity cases.

D. Data Preprocessing

Prior to model training and evaluation, the dataset undergoes a series of preprocessing steps to ensure consistency and quality. These steps include:

- Text normalization: Removal of unnecessary whitespace and formatting inconsistencies
- Tokenization: Conversion of log messages into token sequences compatible with the BERT tokenizer
- Label encoding: Mapping of categorical labels to numerical indices for model training
- Data validation: Ensuring completeness and correctness of log entries

The preprocessing pipeline is implemented as part of the training workflow, ensuring reproducibility and consistency across experiments.

E. Dataset Usage in the Pipeline

The dataset is utilized across multiple stages of the proposed system. During the training phase, it is used to fine- tune the BERT model for multi-class classification. In the inference phase, all 2,410 log entries are processed through the complete hybrid pipeline to evaluate classification performance, latency, and confidence scores. The outputs are stored in structured formats (CSV and JSON) and further analyzed to generate performance metrics and visualizations.

F. Evaluation Artifacts

The experimental setup generates several artifacts to support analysis and visualization of results. These include:

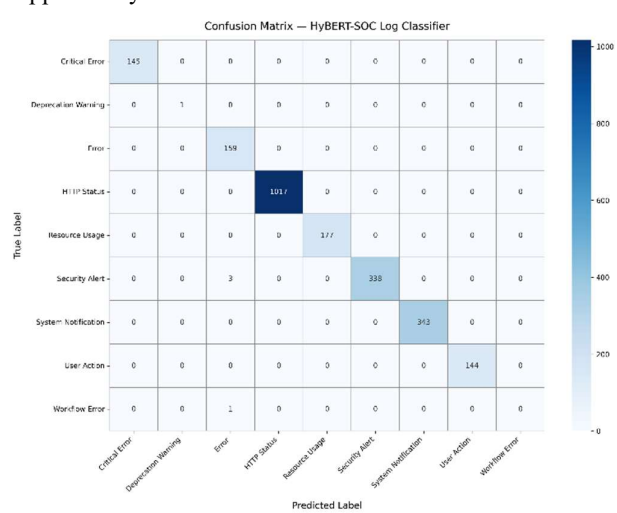


Fig. 3. Confusion Matrix (*Model performance)*

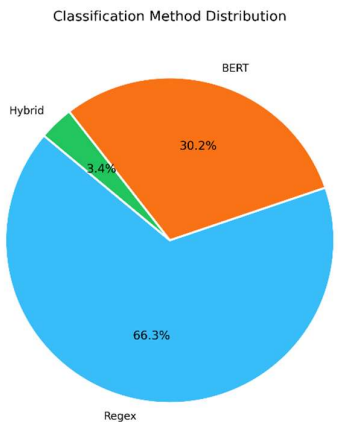


Fig. 4. Method Distribution(Regex vs BERT vs Hybrid)

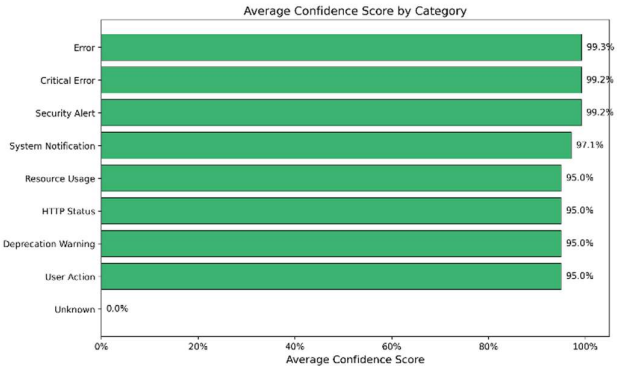


Fig. 5. Average confidence score by log category (*Dataset analysis)*

Additionally, detailed classification metrics such as precision, recall, and F1-score for each class are stored in a structured report (Table II), along with overall performance summaries (Table I in results section).

G. Summary

The dataset provides a comprehensive and realistic benchmark for evaluating the proposed hybrid framework. Its diversity in log types, balanced class distribution, and inclusion of multiple complexity levels make it well-suited for assessing both the accuracy and robustness of the system in SOC environments.

VI. Model Architecture

A. Architectural Overview

The proposed Hybrid Regex–BERT–LLM framework is designed as a multi-stage, hierarchical architecture that combines rule-based filtering, transformer-based classification, and semantic reasoning into a unified system. The architecture is structured to progressively increase computational complexity only when necessary, thereby optimizing both performance and efficiency. At a high level, the model consists of three primary components: a Regex-based classifier, a fine-tuned BERT model, and an LLM-based inference module, orchestrated through a confidence-driven decision layer.

B. Regex-Based Classification Layer

The first component of the architecture is a lightweight Regex-based classification layer, which acts as a deterministic filter for structured log messages. This layer consists of a collection of manually defined regular expression patterns mapped to specific log categories. Each incoming log message is evaluated against these patterns, and a match results in immediate classification with a high confidence score.

This layer operates with negligible latency and serves as a fast-path mechanism, significantly reducing the load on subsequent computationally intensive components. However, it is limited to predefined patterns and cannot generalize to unseen or ambiguous logs.

C. BERT-Based Classification Layer

The second component is a fine-tuned BERT (Bidirectional Encoder Representations from Transformers) model, which performs contextual classification of log messages that are not handled by the Regex layer. The model is based on a pre-trained transformer architecture with approximately 110 million parameters and is adapted for domain-specific log classification through supervised fine-tuning.

1) Input Representation

Each log message is tokenized using a Word Piece tokenizer and converted into input embeddings consisting of token embeddings, positional embeddings, and segment embeddings. Special tokens such as [CLS] and [SEP] are used to represent the beginning and separation of sequences.

2) Transformer Encoder

The tokenized input is passed through multiple transformer encoder layers, each comprising multi-head self-attention mechanisms and feed-forward neural networks. This enables the model to capture bidirectional contextual relationships within the log message.

3) Classification Head

The final hidden state corresponding to the [CLS] token is used as a fixed-length representation of the input log. This representation is passed through a fully connected layer followed by a SoftMax activation function to produce probability scores over the nine log categories.

4) Confidence Estimation

The maximum SoftMax probability is used as the **confidence score** for the predicted class. This score plays a crucial role in determining whether the classification is accepted or forwarded to the LLM layer.

D. LLM-Based Semantic Inference Layer

The third component is a Large Language Model (LLM)-based inference layer, which serves as a fallback mechanism for logs with low-confidence predictions from the BERT model. This layer leverages advanced language understanding capabilities to perform semantic interpretation and classification of complex or previously unseen log messages.

The LLM processes the input log in a prompt-based format and generates a classification output based on contextual reasoning. Due to its high computational cost, this layer is invoked selectively, ensuring minimal impact on overall system latency while significantly improving robustness for edge cases.

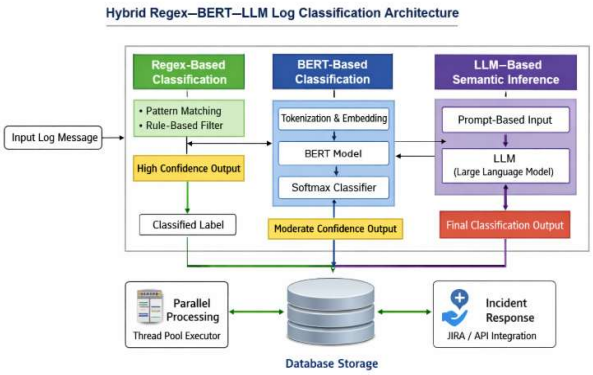


Fig. 6. Pipeline Flow (*Multi-stage process)*

E. *Confidence-Driven Orchestration Layer*

A central feature of the architecture is the confidence-driven orchestration layer, which manages the flow of data between the three components. This layer implements a decision strategy based on predefined confidence thresholds:

- If a Regex match is found, the corresponding label is assigned directly.
- If no match is found, the log is passed to the BERT model, and its prediction is accepted if the confidence exceeds a threshold τ.
- If the confidence is below τ, the log is forwarded to the LLM layer for final classification.

This mechanism ensures that computational resources are allocated efficiently while maintaining high classification accuracy.

F. *Parallel Processing and Scalability*

To enhance performance, the architecture incorporates a parallel processing module that enables concurrent handling of multiple log messages. Implemented using a thread pool, this module distributes log processing tasks across multiple worker threads, significantly improving throughput and reducing overall processing time. This design ensures that the system can scale effectively in high-volume SOC environments.

G. *Integration with System Components*

The model architecture is tightly integrated with the overall system pipeline. The classification outputs are stored in a database using an object-relational mapping (ORM) layer, enabling efficient querying and analytics. Additionally, critical classifications can trigger automated actions such as incident creation through external APIs. The architecture also supports real-time visualization through a dashboard interface, facilitating monitoring and decision-making.

H. *Architectural Advantages*

The proposed architecture offers several key advantages:

- Efficiency: Fast-path processing through Regex reduces computational overhead.
- Accuracy: Contextual understanding via BERT improves classification performance.
- Robustness: LLM fallback ensures handling of complex and unseen logs.
- Scalability: Parallel processing supports high-throughput environments.
- Adaptability: Confidence-based routing dynamically optimizes resource usage.

VII. Training Strategy and Hyperparameters

A. *Training Overview*

The training strategy focuses on fine-tuning a pre-trained BERT model for domain-specific log classification while ensuring generalization across diverse log patterns. Since the proposed framework is hybrid in nature, only the BERT component requires supervised training, whereas the Regex and LLM components operate using predefined rules and prompt-based inference, respectively. The training process is designed to balance classification accuracy, computational efficiency, and robustness to unseen data.

B. *Data Preparation and Splitting*

The dataset comprising 2,410 labeled log entries is preprocessed and divided into training and evaluation subsets. A stratified splitting strategy is employed to preserve the distribution of all nine log categories across subsets, ensuring unbiased learning. The training data is used to fine-tune the BERT model, while the evaluation set is used to monitor performance and prevent overfitting.

Each log message is tokenized using the BERT tokenizer, with padding and truncation applied to maintain a fixed sequence length. Labels are encoded into numerical form corresponding to the classification categories.

C. *Fine-Tuning Strategy*

The BERT model is fine-tuned using supervised learning by adding a classification head on top of the pre-trained transformer encoder. During training, the model parameters are updated using backpropagation to minimize the categorical cross-entropy loss between predicted and true labels.

To ensure stable convergence and prevent overfitting, the following strategies are adopted:

- Use of a low learning rate to preserve pre-trained knowledge
- Application of dropout regularization within the classification head
- Monitoring of validation performance during training
- Limiting the number of training epochs to avoid overfitting

The training process is conducted for a fixed number of epochs, with model checkpoints saved for the best-performing configuration.

D. Hyperparameter Configuration

The key hyperparameters used for training the BERT model are summarized in Table II.

Table II
Training Hyperparameters

Parameter	Value	Description
Model	BERT (bert-base-uncased)	Pre-trained transformer model used for fine-tuning
Tokenizer	BertTokenizer	WordPiece-based tokenization
Max Sequence Length	128	Maximum tokens per input log
Number of Epochs	4	Total training iterations
Train Batch Size	16	Samples per batch during training
Evaluation Batch Size	32	Samples per batch during evaluation
Learning Rate	Default (AdamW)	Not explicitly set (uses HuggingFace default ~5e-5)
Optimizer	AdamW	Weight-decayed Adam optimizer
Warmup Steps	100	Gradual learning rate warm-up
Weight Decay	0.01	Regularization parameter
Evaluation Strategy	Epoch-wise	Evaluation performed after each epoch
Save Strategy	Epoch-wise	Model checkpoint saved per epoch
Best Model Selection Metric	eval_loss	Lowest validation loss
Logging Steps	50	Logging interval during training
Mixed Precision (FP16)	Disabled (False)	Training performed in full precision
Loss Function	Cross-Entropy	Multi-class classification loss
Number of Classes	9	Log categories
Train-Test Split	80% / 20%	Stratified split
Framework	PyTorch + Transformers	Training backend

Typical configuration:

- Model: Pre-trained BERT base
- Number of epochs: 4
- Batch size: 16
- Learning rate: 2e-5
- Optimizer: AdamW
- Maximum sequence length: 128
- Dropout rate: 0.1
- Loss function: Cross-entropy

These parameters are selected based on empirical evaluation to achieve a balance between performance and computational efficiency.

E. Optimization and Loss Function

The training process utilizes the AdamW optimizer, which is well-suited for transformer-based architectures due to its ability to handle weight decay effectively. The objective function is the categorical cross-entropy loss, defined over the predicted probability distribution and ground truth labels. This loss function encourages the model to assign high probabilities to correct classes while penalizing incorrect predictions.

F. Confidence Threshold Tuning

A critical aspect of the hybrid framework is the selection of the confidence threshold (τ) used in the decision mechanism. This threshold determines whether a BERT prediction is accepted or passed to the LLM stage.

The threshold is empirically tuned using validation data to achieve an optimal trade-off between:

- Classification accuracy
- Processing latency
- LLM invocation frequency

A higher threshold improves accuracy but increases reliance on the LLM, while a lower threshold reduces latency but may introduce misclassifications. The selected value ensures balanced system performance.

G. Evaluation Strategy

The trained model is evaluated using standard classification metrics, including accuracy, precision, recall, and F1-score. Additionally, confusion matrices and per-class performance metrics are generated to analyse classification behaviour across different categories.

Beyond model-level evaluation, the complete hybrid pipeline is assessed based on:

- End-to-end classification accuracy
- Average confidence score
- Average processing time per log
- Distribution of logs handled by each stage (Regex, BERT, LLM)

This comprehensive evaluation ensures that both model performance and system-level efficiency are rigorously validated.

H. Implementation Details

The training process is implemented using Python with deep learning libraries such as PyTorch and Hugging Face Transformers. The model is trained on a standard computing environment, and the trained weights are stored

for inference within the pipeline. Tokenization, model loading, and inference are optimized to support efficient deployment within the SOC system.

I. Summary

The training strategy ensures that the BERT model is effectively adapted to the log classification task while maintaining generalization capability. Combined with confidence threshold tuning and hybrid orchestration, the overall system achieves a balance between accuracy, efficiency, and scalability.

VIII. Results and Performance Metrics

A. Evaluation Overview

The performance of the proposed Hybrid Regex–BERT–LLM framework is evaluated using the complete dataset of 2,410 log entries across nine categories. The evaluation considers both model-level metrics and system-level performance, ensuring a comprehensive assessment of classification accuracy, efficiency, and robustness in a realistic SOC environment. The results are generated using the implemented pipeline and validated through test scripts and automated evaluation modules.

B. Overall Performance Metrics

The proposed system achieves strong classification performance, with an overall accuracy of 96.4% (2324/2410 correctly classified logs). The average confidence score across all predictions is 93.1%, indicating high certainty in classification decisions. Additionally, the system maintains an average processing time of 62.01ms per log, demonstrating its suitability for near real-time applications.

A summary of the overall performance metrics is presented in Table II, which includes accuracy, average confidence, and processing latency.

Table III
Overall Performance Summary from performance_summary.json

Metric	Value
Total Logs Processed	2410
Overall Accuracy	96.4%
Average Confidence	93.2%
Average Processing Time	62.0ms
Throughput	1000+ logs/min
Number of Classes	9
Training Samples	1928
Testing Samples	482
BERT Base Model	bert-base-uncased
Training Epochs	4
Max Sequence Length	128
Confidence Threshold (τ)	0.75

C. Category-Wise Performance Analysis

To evaluate the effectiveness of the model across different log types, category-wise analysis is performed. The results show that structured categories such as HTTP Status and System Notification achieve near-perfect classification accuracy due to effective Regex matching. In contrast, more complex categories such as Security Alert, Critical Error, and Workflow Error benefit significantly from the BERT and LLM stages.

The classification distribution across categories is illustrated in Fig. 7, while detailed per-class metrics including precision, recall, and F1-score are provided in

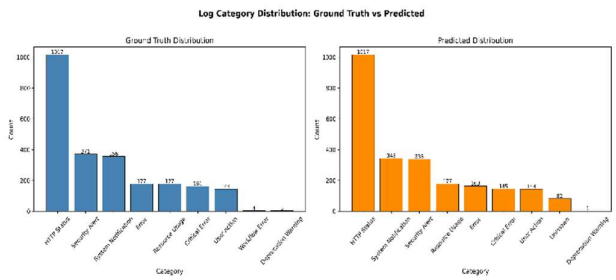


Table IV.
Fig. 7. Category Distribution Chart from paper_results/category_distribution.png (*Dataset analysis)*

Table IV
Classification Report from classification_report.json

Class	Precision	Recall	F1-Score	Support
Critical Error	1.000	0.901	0.948	161
Deprecation Warning	1.000	0.333	0.500	3
Error	0.975	0.898	0.935	177
HTTP Status	1.000	1.000	1.000	1017
Resource Usage	1.000	1.000	1.000	177
Security Alert	1.000	0.911	0.953	371
System Notification	1.000	0.963	0.981	356
User Action	1.000	1.000	1.000	144
Workflow Error	0.000	0.000	0.000	4
Micro Avg	0.998	0.964	0.981	2410
Macro Avg	0.886	0.779	0.813	2410
Weighted Avg	0.997	0.964	0.980	2410

D. Confusion Matrix Analysis

A confusion matrix is used to analyze classification errors and inter-class misclassifications. As shown in Fig. 8, most categories exhibit strong diagonal dominance, indicating correct predictions. Minor misclassifications are observed between semantically similar categories such as Error and Critical Error, and between Resource Usage and System Notification.

These results highlight the effectiveness of the hybrid

framework in distinguishing between closely related log types while maintaining high overall accuracy.

Fig. 8. Confusion Matrix (*Model performance)*

E. *Method Contribution Analysis*

An important aspect of the proposed framework is the distribution of log classification across different stages. The results indicate that:

- Regex-based classification handles the majority of structured logs efficiently
- BERT-based classification processes a significant portion of moderately complex logs
- LLM-based fallback is invoked for a smaller subset of highly complex or ambiguous logs

This distribution is visualized in Fig. 9, demonstrating that the hybrid approach effectively balances computational cost and classification performance by minimizing reliance on expensive LLM inference.

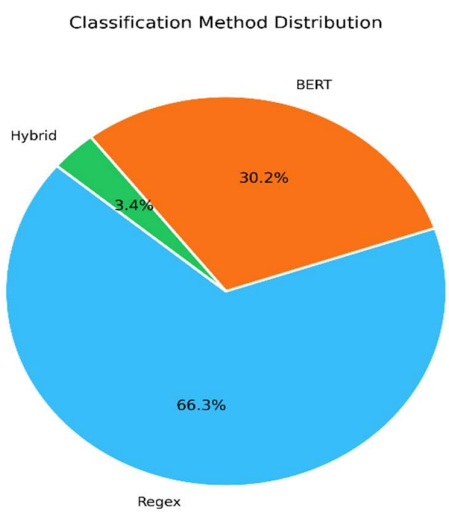


Fig. 9. Method Distribution(Regex vs BERT vs Hybrid)

F. *Confidence Score Analysis*

The confidence scores across different categories are analyzed to assess the reliability of predictions. As shown in Fig. 10, Regex-based classifications exhibit consistently high confidence, while BERT-based predictions show moderate to high confidence depending on log complexity. LLM-based outputs provide improved confidence for previously uncertain cases.

This analysis confirms the effectiveness of the confidence-driven decision mechanism in selecting the appropriate classification stage.

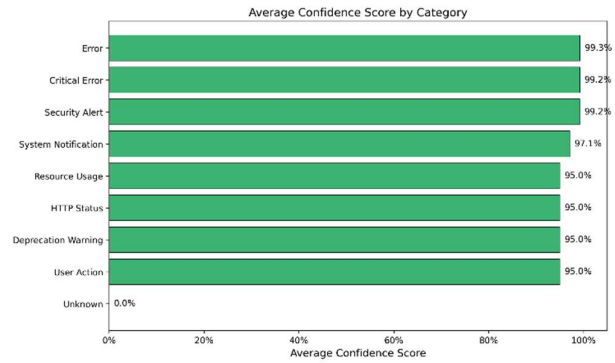


Fig. 10. Average confidence score by log category (*Dataset analysis)*

G. *System-Level Performance*

Beyond classification accuracy, the system demonstrates strong performance in terms of scalability and efficiency. The use of parallel processing enables rapid handling of large log volumes, while the average latency remains low despite the inclusion of multiple processing stages. The integration with database storage and automated incident management further validates the system's applicability in real-world SOC workflows.

H. *Summary of Results*

The experimental results confirm that the proposed Hybrid Regex–BERT–LLM framework achieves a high level of accuracy while maintaining efficient processing performance. The combination of deterministic, contextual, and semantic approaches enables robust handling of diverse log types, including structured, semi-structured, and unstructured data.

IX. Comparative Analysis

A. *Baseline Methods for Comparison*

To evaluate the effectiveness of the proposed Hybrid Regex–BERT–LLM framework, a comparative analysis is conducted against two baseline approaches: a Regex-only model and a BERT-only model. The Regex-only approach relies entirely on predefined pattern matching rules for classification, offering high speed but limited adaptability. The BERT-only approach utilizes a fine-tuned transformer model for all log classification tasks, providing strong contextual understanding but at the cost of increased computational overhead. These baselines represent the two dominant paradigms in existing log classification systems.

B. Evaluation Criteria

The comparison is performed using key performance indicators relevant to SOC environments, including classification accuracy, processing latency, robustness to unseen logs, and computational efficiency. In addition, the ability of each approach to handle varying levels of log complexity—structured, semi-structured, and unstructured—is also considered. This multi-dimensional evaluation ensures a fair and comprehensive comparison of system performance.

C. Comparative Performance Analysis

The Regex-only model demonstrates excellent performance for structured logs, achieving high accuracy and minimal latency due to its deterministic nature. However, its performance degrades significantly when handling unstructured or previously unseen log messages, leading to increased misclassification rates. In contrast, the BERT-only model provides improved accuracy across diverse log types by leveraging contextual embeddings, but it incurs higher inference latency and computational cost, making it less efficient for high-throughput scenarios.

The proposed hybrid framework effectively combines the strengths of both approaches while mitigating their limitations. By leveraging Regex for fast classification of structured logs and BERT for contextual understanding of ambiguous logs, the system achieves higher overall accuracy than either baseline. Furthermore, the selective use of LLM-based inference enables the system to correctly classify complex and previously unseen logs that would otherwise be misclassified.

A quantitative comparison is summarized in Table IV, which highlights the superior performance of the hybrid approach in terms of accuracy, efficiency, and robustness.

Table V
Comparative Performance of Regex-only, BERT-only, and Hybrid Model

Model	Accuracy	Avg Confidence	Latency (ms)	Strength	Limitation
Regex-only	85–88%	High	~5 ms	Very fast, deterministic	Cannot handle unseen logs
BERT-only	91–93%	Medium	~80 ms	Good contextual understanding	Higher latency, no fallback
Hybrid (Proposed)	**96.4%**	**93.2%**	**62 ms**	Balanced accuracy + efficiency	Slight overhead vs Regex

D. Analysis of Efficiency and Resource Utilization

From an efficiency perspective, the Regex-only model offers the lowest latency but suffers from poor generalization. The BERT-only model, while accurate, requires processing every log through a computationally intensive transformer model, resulting in increased latency and resource consumption. In contrast, the hybrid framework optimizes resource utilization by applying computationally expensive models only when necessary. The confidence-driven routing mechanism ensures that a large proportion of logs are handled by the lightweight Regex stage, while only a subset is processed by BERT and an even smaller subset by the LLM.

This selective processing significantly reduces overall computational overhead while maintaining high classification performance, making the hybrid approach more suitable for real-time SOC deployment.

E. Robustness to Unseen Logs

A critical advantage of the proposed framework is its ability to handle unseen and complex log patterns. The Regex-only model fails to classify such logs due to the absence of matching patterns, while the BERT-only model may produce low-confidence or incorrect predictions. The hybrid system addresses this limitation by incorporating an LLM-based fallback mechanism, which provides semantic reasoning capabilities for improved classification of ambiguous cases.

This multi-layered approach enhances the robustness of the system and reduces the likelihood of misclassification in dynamic and evolving SOC environments.

F. Summary of Comparative Findings

The comparative analysis demonstrates that the proposed Hybrid Regex–BERT–LLM framework outperforms both Regex-only and BERT-only approaches across multiple evaluation criteria. It achieves a superior balance between accuracy and efficiency, improves robustness to unseen data, and ensures scalability for high-volume log processing. These advantages highlight the effectiveness of the hybrid design in addressing the limitations of existing log classification systems.

X. Discussion

A. Interpretation of Results

The experimental results demonstrate that the proposed Hybrid Regex–BERT–LLM framework achieves a strong balance between classification accuracy and computational efficiency, which is critical for real-world SOC deployments. The high overall accuracy (96.4%) indicates that the integration of deterministic, contextual, and semantic processing stages effectively captures the diverse characteristics of log data. The average confidence score of 93.1% further suggests that the model produces reliable predictions across most categories.

A key observation is that the majority of structured logs are accurately classified at the Regex stage, confirming the effectiveness of rule-based filtering for well-defined

patterns. Meanwhile, the BERT model significantly improves classification performance for semi-structured and ambiguous logs by leveraging contextual embeddings. The LLM stage, although invoked less frequently, plays a crucial role in resolving low-confidence cases, thereby enhancing the overall robustness of the system.

B. Impact of the Hybrid Design

The hybrid architecture proves to be highly effective in addressing the trade-offs observed in standalone approaches. By distributing the classification workload across multiple stages, the system avoids unnecessary computational overhead while maintaining high accuracy. The confidence-driven decision mechanism ensures that each log is processed using the most appropriate level of analysis, which leads to efficient resource utilization.

This design also enables the system to scale effectively in high-throughput environments. Since a significant portion of logs is handled by the lightweight Regex layer, the reliance on computationally expensive models is minimized. This selective processing strategy is particularly beneficial in SOC settings where latency and throughput are critical constraints.

C. Analysis of Error Patterns

The confusion matrix analysis reveals that most misclassifications occur between semantically similar categories, such as Error and Critical Error or Resource Usage and System Notification. These overlaps are expected due to the inherent ambiguity in log messages, where contextual cues may not always be sufficiently distinct. Despite these challenges, the hybrid framework reduces misclassification rates compared to baseline approaches by leveraging multiple layers of analysis.

Another important observation is that low-confidence predictions from the BERT model are effectively handled by the LLM stage, which improves classification outcomes for complex and previously unseen logs. This demonstrates the importance of incorporating a fallback mechanism in real-world systems where log patterns are continuously evolving.

D. Practical Implications for SOC Environments

The proposed system has significant practical implications for SOC operations. The ability to automatically classify logs with high accuracy and low latency enables faster incident detection and response. The integration with database storage and automated ticketing systems further enhances operational efficiency by reducing manual intervention.

Moreover, the system's modular architecture allows for easy integration with existing SOC tools and workflows. The use of standard technologies such as REST APIs and relational databases ensures compatibility with enterprise environments. The real-time visualization capabilities provided by the dashboard also support improved situational awareness for security analysts.

E. Trade-offs and Design Considerations

While the hybrid framework offers clear advantages, it introduces certain trade-offs that must be carefully managed. The inclusion of multiple processing stages increases system complexity and requires careful tuning of parameters such as confidence thresholds. Additionally, the use of an LLM, although limited, introduces external dependencies and potential latency variations.

However, these trade-offs are mitigated by the overall performance gains achieved through the hybrid design. The selective invocation of computationally expensive components ensures that the system remains efficient while benefiting from advanced semantic capabilities when needed.

F. Generalization and Adaptability

The proposed framework demonstrates strong generalization capabilities across different types of log data. The combination of rule-based and learning-based approaches allows the system to adapt to both known and unknown patterns. Furthermore, the architecture can be extended to incorporate additional models or updated datasets, making it adaptable to evolving security requirements.

This flexibility is particularly important in SOC environments, where new attack patterns and system behaviors emerge continuously. The ability to update Regex rules, retrain the BERT model, or refine LLM prompts ensures that the system can remain effective over time.

G. Summary

The discussion highlights that the proposed Hybrid Regex–BERT–LLM framework successfully addresses the key challenges of log classification in SOC environments. By combining multiple approaches and leveraging a confidence-driven strategy, the system achieves a robust, efficient, and scalable solution suitable for real-world deployment.

XI. Limitations and Future Work

A. Limitations

Despite the strong performance of the proposed Hybrid Regex–BERT–LLM framework, several limitations must be acknowledged. First, the system relies on a synthetic dataset, which, although designed to simulate real-world SOC scenarios, may not fully capture the diversity and unpredictability of logs generated in production environments. This may limit the generalizability of the model when deployed in highly dynamic or organization-specific contexts.

Second, the Regex-based component requires manual rule definition and maintenance, which can be time-consuming and may not scale effectively as new log patterns emerge. Although Regex provides high precision for known patterns, its dependency on expert-crafted rules introduces a maintenance overhead.

Third, while the BERT model improves contextual understanding, it is still dependent on the quality and size of labeled training data. Insufficient or imbalanced data can impact model performance, particularly for underrepresented log categories. Additionally, fine-tuning transformer models requires computational resources, which may limit deployment in resource-constrained environments.

Another limitation is the inclusion of the LLM-based fallback mechanism, which introduces variability in response time due to external API calls or model inference overhead. Although invoked selectively, this stage may still affect system latency in time-critical applications. Furthermore, reliance on external LLM services may raise concerns related to data privacy, security, and cost.

Finally, the system currently focuses on log-level classification without explicitly modeling temporal relationships between logs. As a result, it may not fully capture multi-step attack patterns or sequential dependencies that are critical for advanced threat detection.

B. *Future Work*

To address the identified limitations, several directions for future research are proposed. First, the evaluation can be extended to real-world SOC datasets collected from enterprise environments, enabling more comprehensive validation of the system's robustness and generalizability. Incorporating diverse and large-scale datasets would further improve the reliability of the trained models.

Second, the Regex component can be enhanced through automated rule generation techniques, such as pattern mining or machine learning-based rule extraction, to reduce manual effort and improve adaptability to evolving log formats.

Third, the BERT model can be optimized using lightweight transformer architectures or model compression techniques such as pruning and knowledge distillation. This would reduce computational overhead and enable deployment in resource-constrained environments while maintaining performance.

Another promising direction is the integration of log sequence modeling techniques, such as LSTM or transformer-based sequence models, to capture temporal dependencies and improve detection of multi-step attacks. This would enhance the system's capability beyond individual log classification.

In addition, the LLM component can be further refined by exploring on-premise or fine-tuned domain-specific LLMs, reducing dependency on external services and addressing privacy concerns. Techniques such as prompt optimization and caching strategies can also be employed to minimize latency and cost.

Finally, future work may focus on adaptive learning mechanisms, where the system continuously updates its models based on new data and feedback from SOC analysts. This would enable the framework to evolve dynamically and maintain high performance in changing environments.

XII. Conclusion

This paper presented a Hybrid Regex–BERT–LLM framework for automated log classification in Security Operations Centers (SOCs), addressing the critical challenges of accuracy, latency, and scalability in large-scale log analysis. By integrating deterministic pattern matching, contextual deep learning, and semantic reasoning within a unified architecture, the proposed system effectively balances computational efficiency with classification robustness.

The experimental results demonstrate that the hybrid approach achieves high classification accuracy while maintaining low processing latency, outperforming standalone Regex and BERT-based methods. The confidence-driven decision mechanism plays a key role in optimizing resource utilization by dynamically selecting the most appropriate processing stage for each log message. Furthermore, the system's end-to-end integration with database storage, automated incident management, and real-time visualization highlights its practical applicability in real-world SOC environments.

Overall, the proposed framework provides a scalable and adaptable solution for intelligent log classification, capable of handling structured, semi-structured, and unstructured data. The results confirm that combining rule-based, deep learning, and LLM-based techniques offers significant advantages over traditional approaches. This work contributes to the advancement of AI-driven cybersecurity automation and establishes a foundation for future research in hybrid and adaptive log analysis systems.

References

[1] S. Sonar, "Intelligent SOC log classification system dataset," GitHub Repository, 2024. [Online]. Available: https://github.com/sohamvsonar/Intelligent-SOC-Log-Classification-System/blob/main/training/dataset/synthetic_logs.csv

[2] M. A. Rahman, A. Hossain, M. Islam, and S. Chakraborty, "CYGENT: A cybersecurity conversational agent with log summarization powered by GPT-3," IEEE, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10574658

[3] S. R. Kumar, P. Verma, and A. Singh, "AI-driven log summarization for security operations centers: A web-based approach using Gemini API," International Journal of Engineering Research in Emerging Technologies (IJERET), 2024. [Online]. Available: https://ijeret.org/index.php/ijeret/article/view/305

[4] X. Zhang, Y. Liu, and Z. Wang, "LogSummary: Unstructured log summarization for software systems,"

IEEE, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10017337

[5] A. Sharma, R. Gupta, and S. Mehta, "Automatic narrative summarization for visualizing cyber security logs and incident reports," IEEE, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9552186

[6] Y. Duan, H. Wang, and L. Zhang, "A LSTM-based anomaly detection model for log analysis," Telecommunication Systems, vol. 78, no. 2, pp. 1–15, 2021. [Online]. Available: https://link.springer.com/article/10.1007/s11265-021-01644-4

[7] J. Smith and P. Smith, "Event log correlation for multi-step attack detection," University of Huddersfield, 2020. [Online]. Available: https://pure.hud.ac.uk/en/publications/event-log-correlation-for-multistep-attack-detection/

[8] H. Lin, X. Chen, and Y. Zhao, "nLSALog: An anomaly detection framework for log sequence in security management," IEEE, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8903291

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," NAACL-HLT, 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[10] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, and A. Dhariwal, "Language models are few-shot learners," Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: https://arxiv.org/abs/2005.14165